

Module 3: AI for Mechatronics

Recurrent Neural Networks: From Temporal Context to BPTT

Dr. Rajan Prasad

SMEC
VIT University

Fall 2026

The Same Observation Can Mean Different Things

Situation A

A car is moving at 60 km h^{-1} after its speed increased from 30 to 45 to 60 km h^{-1} .

The driver may still be accelerating.

Situation B

The same car is moving at 60 km h^{-1} after its speed decreased from 90 to 75 to 60 km h^{-1} .

The driver may be braking.

Record A

$$A = [20, 22, 24, 26, 28]$$

Temperature is rising. The motor may be moving toward an abnormal condition.

Record B

$$B = [28, 26, 24, 22, 20]$$

Temperature is falling. The motor may be returning to normal operation.

The current value is identical; the history changes its meaning.

The Same Words, Different Meaning

Consider the following two sentences:

Dog bites man

Man bites dog

Both sentences contain the same words:

{dog, bites, man}.

However, changing their order completely changes the meaning.

How can a model remember which event occurred first and how earlier information affects the current meaning?

The Same Words, Different Meaning

Consider the following two sentences:

Dog bites man

Man bites dog

Both sentences contain the same words:

{dog, bites, man}.

However, changing their order completely changes the meaning.

How can a model remember which event occurred first and how earlier information affects the current meaning?

A Real-Life Conversation

A boy receives the message:

“Let us talk tomorrow.”

Its meaning depends on the earlier conversation:

Positive history:

- “I enjoy spending time with you.”
- “I want to know you better.”
- “Let’s discuss future together.”

Negative history:

- “I consider you only a friend.”
- “I am not interested in a relationship.”
- “I do not see future together.”

Conversation 1: Romantic Context

Boy: You are very special to me.

Girl: I feel the same way.

Boy: “I love you”.

⇒ **Romantic affection**

Conversation 2: Friendship

Boy: You are my best friend.

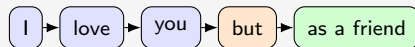
Girl: You always support me.

Boy: “I love you”.

⇒ **Close friendship**

Why Is an RNN Required?

The words “**I love you**” alone do not reveal the complete meaning. Their interpretation depends on the preceding conversation and the words that follow.



An RNN processes the conversation sequentially and carries information from previous words through its hidden state and allowing it to interpret the current words according to their context:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}).$$

Feedforward ANN

$$\hat{\mathbf{y}}_t = f(\mathbf{x}_t; \theta)$$

- Treats each sample as an independent input.
- Has no internal state representing earlier samples.
- Requires a fixed-size input representation.

What the task requires

$$\hat{\mathbf{y}}_t = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots; \theta)$$

The decision should depend on:

- the present measurement;
- the order of past measurements;
- how the system has evolved over time.

At each time step, a motor provides

$$\mathbf{x}_t = \begin{bmatrix} \text{vibration RMS} \\ \text{motor current} \\ \text{temperature} \end{bmatrix}_t.$$

A single high vibration value may arise from impact, load change, or noise.

Diagnostic objective

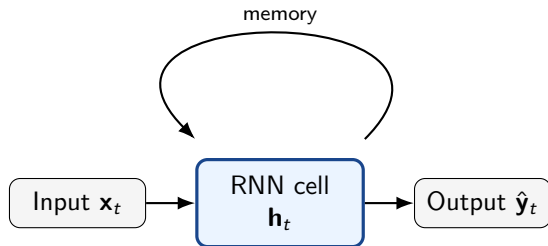
Use the recent measurement history to estimate

$$\hat{y}_t = P(\text{developing fault at } t).$$

Temporal evidence

A gradual rise in vibration and temperature, together with a changing current pattern, is more informative than one isolated sample.

Fault diagnosis is a sequence problem because the evolution of the measurements matters.



Hidden-state update

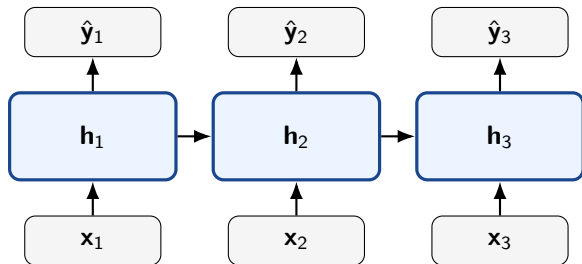
$$\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

Output prediction

$$\hat{\mathbf{y}}_t = g(W_{hy}\mathbf{h}_t + \mathbf{b}_y)$$

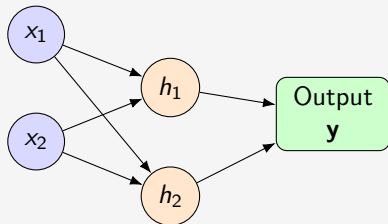
$\mathbf{h}_t$ is a learned summary of the current input and the previous state.

Unrolling Reveals How Information Moves Through Time



- One cell is shown at several time steps.
- The same matrices are reused at every step.
- Forward propagation proceeds from $t = 1$ to T .
- The loss may be calculated at one or every time step.

Artificial Neural Network (ANN)

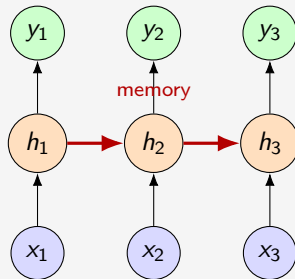


$$y_t = f(x_t)$$

Each observation is processed independently.

Current vibration $\rightarrow$ Condition

Recurrent Neural Network (RNN)



$$h_t = f(x_t, h_{t-1})$$

Information passes from one time step to the next.

Vibration history $\rightarrow$ Condition

Hidden layer

$$\mathbf{a}_t = W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h,$$
$$\mathbf{h}_t = \tanh(\mathbf{a}_t).$$

Output layer

$$\mathbf{z}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y,$$
$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{z}_t).$$

For a binary output, use

$$\hat{y}_t = \sigma(z_t), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

For a sequence-to-sequence problem,

$$L = \sum_{t=1}^T L_t.$$

The initial state is commonly

$$\mathbf{h}_0 = \mathbf{0}.$$

At time t , the term

$$W_{xh}\mathbf{x}_t$$

encodes the current measurement, while

$$W_{hh}\mathbf{h}_{t-1}$$

brings forward the learned summary of earlier measurements.

For motor diagnosis

The hidden state can retain patterns such as:

- sustained vibration growth;
- repeated current spikes;
- temperature rising under similar load;
- correlations across multiple sensors.

The network learns what information to retain because the hidden state affects the loss.

Suppose a fault is correctly recognized only at a later time step.

Which earlier measurements and recurrent connections helped produce that decision?

A gradient must be sent backward through every dependent hidden state.

Backpropagation Through Time

Unroll the RNN across the sequence and apply the chain rule from

$$t = T, T - 1, \dots, 1.$$

Because parameters are shared, their gradient contributions are accumulated over all time steps.

For sigmoid output and binary cross-entropy,

$$L_t = -[y_t \log \hat{y}_t + (1 - y_t) \log(1 - \hat{y}_t)].$$

Define the output pre-activation error:

$$\delta_t^y \equiv \frac{\partial L}{\partial z_t}.$$

Then

$$\boxed{\delta_t^y = \hat{y}_t - y_t}.$$

Since

$$z_t = W_{hy} \mathbf{h}_t + b_y,$$

the time- t contributions are

$$\left. \frac{\partial L}{\partial W_{hy}} \right|_t = \delta_t^y \mathbf{h}_t^T, \quad \left. \frac{\partial L}{\partial b_y} \right|_t = \delta_t^y.$$

The error sent to the hidden state is

$$W_{hy}^T \delta_t^y.$$

BPTT Step 2: Two Error Paths Meet at $\mathbf{h}_t$

Current-output path

Because $\mathbf{h}_t$ produces $\hat{\mathbf{y}}_t$,

$$\left. \frac{\partial L}{\partial \mathbf{h}_t} \right|_{\text{output}} = W_{hy}^T \delta_t^y.$$

Future-state path

Since

$$\mathbf{a}_{t+1} = W_{xh} \mathbf{x}_{t+1} + W_{hh} \mathbf{h}_t + \mathbf{b}_h,$$

the future contributes

$$\left. \frac{\partial L}{\partial \mathbf{h}_t} \right|_{\text{future}} = W_{hh}^T \delta_{t+1}^h.$$

$$\frac{\partial L}{\partial \mathbf{h}_t} = W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h$$

The hidden activation is

$$\mathbf{h}_t = \tanh(\mathbf{a}_t).$$

Elementwise,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t} = 1 - \mathbf{h}_t^2.$$

Define

$$\delta_t^h \equiv \frac{\partial L}{\partial \mathbf{a}_t}.$$

Applying the chain rule gives the central BPTT recurrence:

$$\delta_t^h = (W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h) \odot (1 - \mathbf{h}_t^2)$$

with the boundary condition

$$\delta_{T+1}^h = \mathbf{0}.$$

The hidden error combines current output error, future error, and the activation derivative.

For sigmoid output with binary cross-entropy,

$$\frac{\partial L_t}{\partial \hat{y}_t} = \frac{\hat{y}_t - y_t}{\hat{y}_t(1 - \hat{y}_t)}.$$

The sigmoid derivative is

$$\frac{\partial \hat{y}_t}{\partial z_t} = \hat{y}_t(1 - \hat{y}_t).$$

Applying the chain rule,

$$\begin{aligned}\delta_t^y &\equiv \frac{\partial L_t}{\partial z_t} \\ &= \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \\ &= \boxed{\hat{y}_t - y_t}.\end{aligned}$$

Interpretation

$\delta_t^y < 0$ increases the output; $\delta_t^y > 0$ decreases it during gradient descent.

Step 2: Output-Layer Gradients

The output pre-activation is

$$\mathbf{z}_t = W_{hy} \mathbf{h}_t + \mathbf{b}_y.$$

At one time step,

$$\left. \frac{\partial L}{\partial W_{hy}} \right|_t = \delta_t^y \mathbf{h}_t^\top, \quad \left. \frac{\partial L}{\partial \mathbf{b}_y} \right|_t = \delta_t^y.$$

Since the output parameters are shared,

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T \delta_t^y \mathbf{h}_t^\top$$

$$\frac{\partial L}{\partial \mathbf{b}_y} = \sum_{t=1}^T \delta_t^y.$$

The error sent from the output to the hidden state is $W_{hy}^\top \delta_t^y$.

BPTT Step 4: Derive the Hidden-Layer Gradients

At one time step,

$$\begin{aligned}\left. \frac{\partial L}{\partial W_{xh}} \right|_t &= \delta_t^h \mathbf{x}_t^\top, \\ \left. \frac{\partial L}{\partial W_{hh}} \right|_t &= \delta_t^h \mathbf{h}_{t-1}^\top, \\ \left. \frac{\partial L}{\partial \mathbf{b}_h} \right|_t &= \delta_t^h.\end{aligned}$$

The same parameters are reused at every time step. Therefore,

$$\begin{aligned}\frac{\partial L}{\partial W_{xh}} &= \sum_{t=1}^T \delta_t^h \mathbf{x}_t^\top, \\ \frac{\partial L}{\partial W_{hh}} &= \sum_{t=1}^T \delta_t^h \mathbf{h}_{t-1}^\top, \\ \frac{\partial L}{\partial \mathbf{b}_h} &= \sum_{t=1}^T \delta_t^h.\end{aligned}$$

Parameter sharing across time produces a sum of gradient contributions.

Backward error recursion

$$\delta_t^y = \hat{y}_t - y_t$$

$$\delta_t^h = \left(W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h \right) \odot (1 - \mathbf{h}_t^2)$$

$$\delta_{T+1}^h = \mathbf{0}$$

Accumulated parameter gradients

$$\frac{\partial L}{\partial W_{hy}} = \sum_t \delta_t^y \mathbf{h}_t^T,$$

$$\frac{\partial L}{\partial \mathbf{b}_y} = \sum_t \delta_t^y,$$

$$\frac{\partial L}{\partial W_{xh}} = \sum_t \delta_t^h \mathbf{x}_t^T,$$

$$\frac{\partial L}{\partial W_{hh}} = \sum_t \delta_t^h \mathbf{h}_{t-1}^T,$$

$$\frac{\partial L}{\partial \mathbf{b}_h} = \sum_t \delta_t^h.$$

Use the sequence

$$(x_1, x_2) = (1, 0), \quad (y_1, y_2) = (1, 0),$$

and initial state $h_0 = 0$.

Parameters

$$W_{xh} = 0.5, \quad W_{hh} = 0.2, \quad W_{hy} = 0.8, \\ b_h = b_y = 0.$$

For each time step,

$$a_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h,$$

$$h_t = \tanh(a_t),$$

$$z_t = W_{hy}h_t + b_y,$$

$$\hat{y}_t = \sigma(z_t).$$

The total binary cross-entropy loss is

$$L = L_1 + L_2.$$

Time step $t = 1$

$$\begin{aligned}a_1 &= 0.5(1) + 0.2(0) = 0.5000, \\h_1 &= \tanh(0.5000) = 0.4621, \\z_1 &= 0.8(0.4621) = 0.3697, \\\hat{y}_1 &= \sigma(0.3697) = 0.5914.\end{aligned}$$

Time step $t = 2$

$$\begin{aligned}a_2 &= 0.5(0) + 0.2(0.4621) = 0.0924, \\h_2 &= \tanh(0.0924) = 0.0922, \\z_2 &= 0.8(0.0922) = 0.0737, \\\hat{y}_2 &= \sigma(0.0737) = 0.5184.\end{aligned}$$

$$L = -\log(0.5914) - \log(1 - 0.5184) = 1.2560.$$

At $t = 1$,

$$\delta_1^y = \hat{y}_1 - y_1 = 0.5914 - 1 = -0.4086.$$

A negative value means the prediction should increase toward the target 1.

At $t = 2$,

$$\delta_2^y = \hat{y}_2 - y_2 = 0.5184 - 0 = 0.5184.$$

A positive value means the prediction should decrease toward the target 0.

Output-layer bias gradient

$$\frac{\partial L}{\partial b_y} = \delta_1^y + \delta_2^y = -0.4086 + 0.5184 = 0.1098.$$

Start at $t = 2$

Since $\delta_3^h = 0$,

$$\begin{aligned}\delta_2^h &= (0.8\delta_2^y)(1 - h_2^2) \\ &= [0.8(0.5184)](1 - 0.0922^2) \\ &= 0.4112.\end{aligned}$$

Continue to $t = 1$

$$\begin{aligned}\delta_1^h &= (0.8\delta_1^y + 0.2\delta_2^h)(1 - h_1^2) \\ &= [0.8(-0.4086) + 0.2(0.4112)](1 - 0.4621^2) \\ &= -0.1924.\end{aligned}$$

δ_1^h contains both the error at $t = 1$ and error arriving from $t = 2$.

$$\begin{aligned}\frac{\partial L}{\partial W_{hy}} &= \delta_1^y h_1 + \delta_2^y h_2 \\ &= (-0.4086)(0.4621) + (0.5184)(0.0922) \\ &= -0.1410,\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W_{xh}} &= \delta_1^h x_1 + \delta_2^h x_2 \\ &= (-0.1924)(1) + (0.4112)(0) \\ &= -0.1924.\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W_{hh}} &= \delta_1^h h_0 + \delta_2^h h_1 \\ &= 0 + (0.4112)(0.4621) = 0.1900,\end{aligned}$$

$$\frac{\partial L}{\partial b_h} = \delta_1^h + \delta_2^h = 0.2188,$$

$$\frac{\partial L}{\partial b_y} = 0.1098.$$

Gradient-descent update

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}.$$

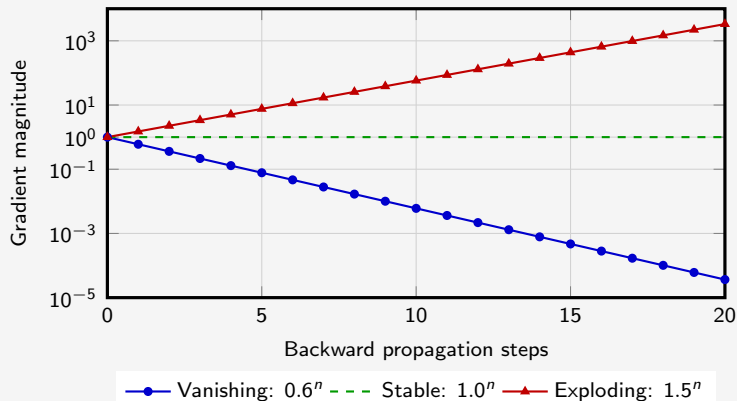
Forward pass: $t = 1 \rightarrow T$

- 1 Set $\mathbf{h}_0 = \mathbf{0}$.
- 2 Compute $\mathbf{a}_t$ and $\mathbf{h}_t$.
- 3 Compute $\mathbf{z}_t$, $\hat{\mathbf{y}}_t$, and L_t .
- 4 Store $\mathbf{x}_t$, $\mathbf{h}_{t-1}$, $\mathbf{h}_t$, and $\hat{\mathbf{y}}_t$.
- 5 Sum the sequence loss $L = \sum_t L_t$.

Backward pass: $t = T \rightarrow 1$

- 1 Set $\delta_{T+1}^h = \mathbf{0}$ and all gradients to zero.
- 2 Compute δ_t^y .
- 3 Compute δ_t^h using the recurrence.
- 4 Add the time- t contribution to each gradient.
- 5 Optionally clip gradients, then update the parameters.

Gradient Magnitude During Backpropagation



Vanishing Gradient

If the effective derivative is less than one,

$$0.6^{20} \approx 3.7 \times 10^{-5}.$$

The gradient approaches zero, so early time steps receive almost no learning information.

Exploding Gradient

If the effective derivative is greater than one,

$$1.5^{20} \approx 3.3 \times 10^3.$$

The gradient grows rapidly and causes unstable parameter

Gradient Propagation Through Time

To transfer an error from time T to an earlier time t , the gradient repeatedly passes through the recurrent connection:

$$\frac{\partial L}{\partial \mathbf{h}_t} \propto \prod_{k=t+1}^T [W_{hh}^T \text{diag}(1 - \mathbf{h}_k^2)] .$$

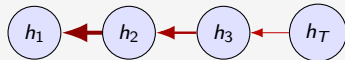
Therefore, the gradient contains a product of many recurrent-weight and activation derivatives.

Main Cause

Repeated multiplication can make the gradient **progressively smaller** or **progressively larger**.

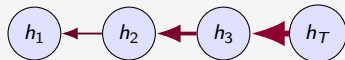
Gradient Behaviour

Vanishing gradient



The error becomes weaker as it moves backward.

Exploding gradient



The error becomes excessively large as it moves backward.

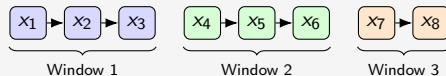
1. Control the Gradient

- **Gradient clipping:** restrict excessively large gradients before updating the parameters:

$$\mathbf{g} \leftarrow \mathbf{g} \min \left(1, \frac{\tau}{\|\mathbf{g}\|} \right).$$

- **LSTM or GRU:** use gated memory units to learn longer-term dependencies.
- **Weight initialization:** initialize recurrent weights carefully to improve gradient propagation.

2. Use Truncated BPTT



A long sequence, divided into shorter windows and thus gradients are propagated only within each window. It reduces computational cost and limits very long gradient chains.

3. Prepare the Sensor Data

- Normalize features, use consistent sampling interval, separate operating modes if their dynamics differ significantly.

Recommended Training Workflow

Normalize data → Create short sequences → Train with clipping → Validate on unseen operation

Monitoring and diagnosis

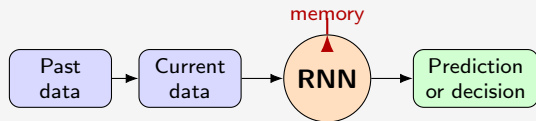
- bearing and motor fault classification;
- anomaly detection from sensor streams;
- remaining useful life estimation.

Motion and interaction

- robot trajectory prediction;
- human activity and gesture recognition;
- force and torque sequence modelling;
- time-series forecasting for control support.

Use an RNN when the order and temporal context of measurements influence the desired output.

What Can an RNN Learn?



An RNN can learn:

- temporal patterns and trends;
- relationships between earlier and current events;
- changes that develop over time;
- likely future states of a system.

Sequence → Context → Decision

Major Real-Life Applications

Speech
voice recognition

Manufacturing
predictive maintenance

Healthcare
ECG monitoring

Vehicles
motion prediction

Energy
load forecasting

Security
anomaly detection

Motor current, vibration and temperature are observed over time to detect degradation before machine failure.

Sensor History → Fault Prediction

RNNs transform continuously changing data into timely predictions, classifications and control decisions.

Forward in time

$$\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

The hidden state carries learned temporal context.

Backward in time

$$\delta_t^h = (W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h) \odot (1 - \mathbf{h}_t^2)$$

BPTT assigns later errors to earlier states and shared parameters.

Scan to Open : Colab Notebook



An RNN learns temporal relationships by carrying state forward and propagating error backward.

Instructions

- Scan the QR code to access the quiz.
- Sign in using your **VIT Google account**.
- The quiz contains **10 questions**.
- Only **one submission per account** is allowed.

QUIZ: Scan to begin



Thank You

Questions?

Hidden Layer

At time step t , the hidden pre-activation is

$$\mathbf{a}_t = W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h.$$

The hidden state is

$$\mathbf{h}_t = \tanh(\mathbf{a}_t)$$

Thus, $\mathbf{h}_t$ depends on:

- the current input $\mathbf{x}_t$;
- the previous hidden state $\mathbf{h}_{t-1}$.

Output Layer

The output pre-activation is

$$z_t = W_{hy}\mathbf{h}_t + b_y.$$

For binary classification,

$$\hat{y}_t = \sigma(z_t)$$

where

$$\sigma(z_t) = \frac{1}{1 + e^{-z_t}}.$$

Training Objective

Calculate how the total loss changes with every weight and bias.

For the target y_t and prediction $\hat{y}_t$, the binary cross-entropy loss at time t is

$$L_t = -[y_t \log \hat{y}_t + (1 - y_t) \log(1 - \hat{y}_t)]$$

The total sequence loss is

$$L = \sum_{t=1}^T L_t$$

Backward Calculation

At each time step, the gradient passes through

$$L_t \longrightarrow \hat{y}_t \longrightarrow z_t \longrightarrow \mathbf{h}_t \longrightarrow \mathbf{a}_t.$$

The hidden-state error also propagates backward through time:

$$\mathbf{h}_{t+1} \longrightarrow \mathbf{h}_t \longrightarrow \mathbf{h}_{t-1}.$$

BPTT

Backpropagation Through Time applies the chain rule across both network layers and time steps.

Step 1: Differentiate the Loss with Respect to Prediction

Starting from

$$L_t = -[y_t \log \hat{y}_t + (1 - y_t) \log(1 - \hat{y}_t)],$$

differentiate the first term:

$$\frac{\partial}{\partial \hat{y}_t} [-y_t \log \hat{y}_t] = -\frac{y_t}{\hat{y}_t}.$$

Differentiate the second term:

$$\begin{aligned} \frac{\partial}{\partial \hat{y}_t} [-(1 - y_t) \log(1 - \hat{y}_t)] \\ = -(1 - y_t) \frac{-1}{1 - \hat{y}_t} \\ = \frac{1 - y_t}{1 - \hat{y}_t}. \end{aligned}$$

Therefore,

$$\boxed{\frac{\partial L_t}{\partial \hat{y}_t} = -\frac{y_t}{\hat{y}_t} + \frac{1 - y_t}{1 - \hat{y}_t}}$$

Step 2: Differentiate the Sigmoid Output

The sigmoid output is

$$\hat{y}_t = \sigma(z_t) = \frac{1}{1 + e^{-z_t}}.$$

Differentiate with respect to z_t :

$$\begin{aligned}\frac{\partial \hat{y}_t}{\partial z_t} &= \frac{\partial}{\partial z_t} (1 + e^{-z_t})^{-1} \\ &= -(1 + e^{-z_t})^{-2} (-e^{-z_t}) \\ &= \frac{e^{-z_t}}{(1 + e^{-z_t})^2}.\end{aligned}$$

Because

$$\hat{y}_t = \frac{1}{1 + e^{-z_t}}$$

and

$$1 - \hat{y}_t = \frac{e^{-z_t}}{1 + e^{-z_t}},$$

we obtain

$$\boxed{\frac{\partial \hat{y}_t}{\partial z_t} = \hat{y}_t(1 - \hat{y}_t)}$$

Next Step

Combine the loss derivative and sigmoid derivative using the chain rule.

Step 3: Derive the Output Pre-activation Error

Define the output pre-activation error:

$$\delta_t^y \equiv \frac{\partial L_t}{\partial z_t}.$$

Using the chain rule,

$$\delta_t^y = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t}.$$

Substituting the two derivatives,

Simplifying,

$$\delta_t^y = \left[-\frac{y_t}{\hat{y}_t} + \frac{1 - y_t}{1 - \hat{y}_t} \right] \hat{y}_t (1 - \hat{y}_t).$$

Expanding,

$$\delta_t^y = -y_t(1 - \hat{y}_t) + (1 - y_t)\hat{y}_t.$$

$$\begin{aligned} \delta_t^y &= -y_t + y_t\hat{y}_t + \hat{y}_t - y_t\hat{y}_t \\ &= \hat{y}_t - y_t. \end{aligned}$$

Therefore,

$$\delta_t^y = \frac{\partial L_t}{\partial z_t} = \hat{y}_t - y_t$$

Step 4: Derive the Output-Weight Gradient

The output pre-activation is

$$z_t = W_{hy} \mathbf{h}_t + b_y.$$

The time- t contribution to the output-weight gradient is

$$\left. \frac{\partial L}{\partial W_{hy}} \right|_t = \frac{\partial L}{\partial z_t} \frac{\partial z_t}{\partial W_{hy}}.$$

Because

$$\frac{\partial L}{\partial z_t} = \delta_t^y$$

and

$$\frac{\partial z_t}{\partial W_{hy}} = \mathbf{h}_t^T,$$

the time- t contribution is

$$\left. \frac{\partial L}{\partial W_{hy}} \right|_t = \delta_t^y \mathbf{h}_t^T$$

Since W_{hy} is shared across all time steps,

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T \delta_t^y \mathbf{h}_t^T$$

Parameter Sharing

Every time step contributes to the gradient of the same output weight matrix.

Step 5: Derive the Output-Bias Gradient

From

$$z_t = W_{hy} \mathbf{h}_t + b_y,$$

the derivative of z_t with respect to b_y is

$$\frac{\partial z_t}{\partial b_y} = 1.$$

Therefore,

$$\begin{aligned} \left. \frac{\partial L}{\partial b_y} \right|_t &= \frac{\partial L}{\partial z_t} \frac{\partial z_t}{\partial b_y} \\ &= \delta_t^y (1). \end{aligned}$$

The time- t contribution is

$$\left. \frac{\partial L}{\partial b_y} \right|_t = \delta_t^y$$

Summing over the complete sequence,

$$\frac{\partial L}{\partial b_y} = \sum_{t=1}^T \delta_t^y$$

Using $\delta_t^y = \hat{y}_t - y_t$,

$$\frac{\partial L}{\partial b_y} = \sum_{t=1}^T (\hat{y}_t - y_t)$$

Step 6: Error Passed from Output to Hidden State

The output layer depends on the hidden state through

$$z_t = W_{hy} \mathbf{h}_t + b_y.$$

The loss gradient reaching $\mathbf{h}_t$ from the current output is

$$\left. \frac{\partial L}{\partial \mathbf{h}_t} \right|_{\text{output}} = \left(\frac{\partial z_t}{\partial \mathbf{h}_t} \right)^T \frac{\partial L}{\partial z_t}.$$

Since

$$\frac{\partial z_t}{\partial \mathbf{h}_t} = W_{hy},$$

we obtain

$$\left. \frac{\partial L}{\partial \mathbf{h}_t} \right|_{\text{output}} = W_{hy}^T \delta_t^y$$

However, $\mathbf{h}_t$ also affects the next hidden state:

$$\mathbf{h}_t \longrightarrow \mathbf{a}_{t+1} \longrightarrow \mathbf{h}_{t+1} \longrightarrow L_{t+1}, L_{t+2}, \dots$$

Therefore, the complete hidden-state gradient must include:

$$\text{current output error} + \text{future recurrent error}$$

Step 7: Complete Gradient Reaching the Hidden State

The hidden state $\mathbf{h}_t$ influences the total loss through two paths:

- 1 the output at time t ;
- 2 the hidden state at time $t + 1$.

The current-output contribution is

$$W_{hy}^T \delta_t^y.$$

The future recurrent contribution is

$$W_{hh}^T \delta_{t+1}^h.$$

Therefore,

$$\frac{\partial L}{\partial \mathbf{h}_t} = W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h$$

where

$$\delta_{t+1}^h \equiv \frac{\partial L}{\partial \mathbf{a}_{t+1}}.$$

At the final time step, there is no future hidden state. Hence,

$$\delta_{T+1}^h = \mathbf{0}$$

Step 8: Differentiate Through the Tanh Activation

The hidden state is

$$\mathbf{h}_t = \tanh(\mathbf{a}_t).$$

For a scalar variable a ,

$$\frac{d}{da} \tanh(a) = 1 - \tanh^2(a).$$

Therefore, elementwise,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t} = 1 - \mathbf{h}_t^2.$$

Define

$$\delta_t^h \equiv \frac{\partial L}{\partial \mathbf{a}_t}.$$

Using the chain rule,

$$\delta_t^h = \frac{\partial L}{\partial \mathbf{h}_t} \odot \frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t}.$$

Substituting both terms gives

$$\delta_t^h = (W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h) \odot (1 - \mathbf{h}_t^2)$$

Here, $\odot$ denotes elementwise multiplication.

Step 9: Input-to-Hidden Weight Gradient

The hidden pre-activation is

$$\mathbf{a}_t = W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h.$$

The time- t contribution is

$$\left. \frac{\partial L}{\partial W_{xh}} \right|_t = \frac{\partial L}{\partial \mathbf{a}_t} \frac{\partial \mathbf{a}_t}{\partial W_{xh}}.$$

Because

$$\frac{\partial L}{\partial \mathbf{a}_t} = \delta_t^h,$$

and the local derivative supplies $\mathbf{x}_t^T$,

$$\left. \frac{\partial L}{\partial W_{xh}} \right|_t = \delta_t^h \mathbf{x}_t^T$$

Since W_{xh} is shared across time,

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \delta_t^h \mathbf{x}_t^T$$

Dimensions

If $\delta_t^h \in \mathbb{R}^{n_h}$ and $\mathbf{x}_t \in \mathbb{R}^{n_x}$, the outer product has size $n_h \times n_x$.

Step 10: Recurrent-Weight Gradient

The recurrent contribution to the hidden pre-activation is

$$W_{hh}\mathbf{h}_{t-1}.$$

Therefore,

$$\left. \frac{\partial L}{\partial W_{hh}} \right|_t = \delta_t^h \mathbf{h}_{t-1}^\top.$$

The same recurrent matrix is used at every time step.

Summing the contributions,

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \delta_t^h \mathbf{h}_{t-1}^\top$$

At $t = 1$,

$$\left. \frac{\partial L}{\partial W_{hh}} \right|_{t=1} = \delta_1^h \mathbf{h}_0^\top.$$

If

$$\mathbf{h}_0 = \mathbf{0},$$

then the first recurrent-weight contribution is zero.

From

$$\mathbf{a}_t = W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h,$$

we have

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{b}_h} = \mathbf{I}.$$

Therefore,

$$\left. \frac{\partial L}{\partial \mathbf{b}_h} \right|_t = \delta_t^h.$$

Summing over all time steps,

$$\frac{\partial L}{\partial \mathbf{b}_h} = \sum_{t=1}^T \delta_t^h$$

Why Do We Sum?

The same bias vector $\mathbf{b}_h$ is used to calculate every hidden state:

$$\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T.$$

Therefore, every time step contributes to its gradient.

Forward Pass

$$\mathbf{a}_t = W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t)$$

$$z_t = W_{hy}\mathbf{h}_t + b_y$$

$$\hat{y}_t = \sigma(z_t)$$

$$L = \sum_{t=1}^T L_t$$

Backward Error Equations

$$\delta_t^y = \hat{y}_t - y_t$$

$$\delta_t^h = \left(W_{hy}^T \delta_t^y + W_{hh}^T \delta_{t+1}^h \right) \odot (1 - \mathbf{h}_t^2)$$

$$\delta_{T+1}^h = \mathbf{0}.$$

Parameter Gradients

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T \delta_t^y \mathbf{h}_t^T$$

$$\frac{\partial L}{\partial b_y} = \sum_{t=1}^T \delta_t^y$$

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \delta_t^h \mathbf{x}_t^T$$

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \delta_t^h \mathbf{h}_{t-1}^T$$

$$\frac{\partial L}{\partial \mathbf{b}_h} = \sum_{t=1}^T \delta_t^h$$

Forward Pass: $t = 1$ to T

① Initialize $\mathbf{h}_0$.

② Calculate

$$\mathbf{a}_t, \quad \mathbf{h}_t, \quad \mathbf{z}_t, \quad \hat{y}_t.$$

③ Calculate L_t .

④ Store all intermediate values.

⑤ Calculate

$$L = \sum_{t=1}^T L_t.$$

Backward Pass: $t = T$ to 1

① Initialize

$$\delta_{T+1}^h = \mathbf{0}.$$

② Calculate

$$\delta_t^y = \hat{y}_t - y_t.$$

③ Calculate δ_t^h .

④ Accumulate all parameter gradients.

⑤ Update each parameter:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$